

Docker Command Cheat Sheet

The commands you'll actually use, grouped by task

Images · Containers · Networks · Volumes · Compose · Cleanup

1. Check your setup

Command	What it does
<code>docker --version</code>	Show the Docker version
<code>docker compose version</code>	Show the Compose version
<code>docker info</code>	Details about the Docker engine (fails if Docker isn't running)
<code>docker run --rm hello-world</code>	Quick test that everything works

2. Images

Command	What it does
<code>docker images</code>	List local images (same as <code>docker image ls</code>)
<code>docker pull mysql:8.4</code>	Download an image from Docker Hub
<code>docker build -t myapp .</code>	Build an image from the Dockerfile in the current folder and name it myapp
<code>docker build -t myapp:1.0 ./DockerBackend</code>	Build from another folder (the build context), with a version tag
<code>docker build -f other.Dockerfile .</code>	Use a Dockerfile with a different name
<code>docker build --no-cache -t myapp .</code>	Rebuild everything, ignoring cached layers
<code>docker build --build-arg KEY=value .</code>	Set an ARG value for this build
<code>docker build --target build .</code>	Build only up to a named stage (multi-stage builds)
<code>docker history myapp</code>	Show the image's layers and their sizes
<code>docker image inspect myapp</code>	Full details as JSON (env vars, entrypoint, ports...)
<code>docker tag myapp user/myapp:1.0</code>	Give an image another name/tag
<code>docker rmi myapp</code>	Delete an image (no container may be using it)

3. Run containers

Flags can be combined, for example: `docker run -d --name backend --network lesson-net -p 8080:8080 -e DB_HOST=db docker-backend`

Command	What it does
<code>docker run myapp</code>	Create and start a container (in the foreground)
<code>docker run -d myapp</code>	Run in the background (detached)
<code>docker run --name web myapp</code>	Give the container a name (otherwise it gets a random one)
<code>docker run -p 8080:8080 myapp</code>	Publish a port, HOST:CONTAINER
<code>docker run -e KEY=value myapp</code>	Set an environment variable
<code>docker run --env-file .env myapp</code>	Load environment variables from a file
<code>docker run -v mydata:/data myapp</code>	Mount a named volume at /data
<code>docker run -v "\$(pwd)":/app myapp</code>	Mount a folder from your computer (bind mount)

Command	What it does
<code>docker run --network mynet myapp</code>	Attach the container to a network
<code>docker run --rm myapp</code>	Delete the container automatically when it stops
<code>docker run -it alpine sh</code>	Interactive terminal: get a shell in a fresh container
<code>docker run --restart unless-stopped myapp</code>	Restart policy (no / on-failure / always / unless-stopped)

4. Manage containers

Command	What it does
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers, including stopped ones
<code>docker stop web</code>	Stop gracefully (sends SIGTERM, then SIGKILL after 10 s)
<code>docker start web</code>	Start a stopped container again
<code>docker restart web</code>	Stop + start
<code>docker kill web</code>	Stop immediately
<code>docker rm web</code>	Delete a stopped container
<code>docker rm -f web</code>	Stop and delete in one step
<code>docker rename web web2</code>	Rename a container

5. Look inside: logs, shells, debugging

Command	What it does
<code>docker logs web</code>	Show the container's output
<code>docker logs -f web</code>	Follow the logs live (Ctrl+C to stop following)
<code>docker logs --tail 50 web</code>	Only the last 50 lines
<code>docker exec -it web sh</code>	Open a shell inside a running container (bash if the image has it)
<code>docker exec web env</code>	Run one command inside, e.g. print the environment variables
<code>docker inspect web</code>	Full details: IP address, mounts, env, health status...
<code>docker inspect --format '{{.State.Health.Status}}' web</code>	Show just the health status
<code>docker stats</code>	Live CPU / memory usage of each container
<code>docker top web</code>	Processes running in the container
<code>docker port web</code>	Show the published ports
<code>docker cp web:/app/file.txt .</code>	Copy a file out of a container (or the other way round)

6. Networks

Command	What it does
<code>docker network ls</code>	List networks
<code>docker network create mynet</code>	Create a user-defined bridge network (containers can reach each other by name)
<code>docker network inspect mynet</code>	See which containers are attached and their IPs
<code>docker network connect mynet web</code>	Attach a running container to a network
<code>docker network disconnect mynet web</code>	Detach it
<code>docker network rm mynet</code>	Delete a network

7. Volumes

Command	What it does
<code>docker volume ls</code>	List volumes
<code>docker volume create mydata</code>	Create a named volume
<code>docker volume inspect mydata</code>	Show where the volume is stored and other details
<code>docker volume rm mydata</code>	Delete a volume, and all data in it
<code>docker volume prune</code>	Delete all volumes no container is using

8. Docker Compose

Command	What it does
<code>docker compose up</code>	Create and start all services in docker-compose.yml
<code>docker compose up --build</code>	Rebuild images first (use this after code changes)
<code>docker compose up -d</code>	Start in the background
<code>docker compose up --build backend</code>	Rebuild and start only one service (plus what it depends on)
<code>docker compose ps</code>	List services with status, health and ports
<code>docker compose logs -f</code>	Follow the logs of all services
<code>docker compose logs -f backend</code>	Follow the logs of one service
<code>docker compose exec backend sh</code>	Shell inside a running service
<code>docker compose run --rm backend sh</code>	Start a new one-off container for a service
<code>docker compose stop</code>	Stop the services (containers are kept)
<code>docker compose start</code>	Start stopped services again
<code>docker compose restart backend</code>	Restart one service
<code>docker compose down</code>	Stop and remove containers + networks. Volumes are kept.
<code>docker compose down -v</code>	Same, and delete volumes (database data is lost)
<code>docker compose down --rmi all</code>	Also delete the images Compose built

Command	What it does
<code>docker compose build</code>	Only build the images, don't start anything
<code>docker compose build --no-cache</code>	Build from scratch
<code>docker compose pull</code>	Download newer versions of the <code>image: services</code>
<code>docker compose config</code>	Print the final config with all <code>\${VARIABLES}</code> filled in (great for debugging)
<code>docker compose top</code>	Processes running in each service
<code>docker compose -f a.yml -f b.yml up</code>	Combine several compose files (the later one overrides the earlier)

9. Cleanup

Command	What it does
<code>docker container prune</code>	Delete all stopped containers
<code>docker image prune</code>	Delete dangling images (untagged leftovers from rebuilds)
<code>docker image prune -a</code>	Delete all images not used by a container
<code>docker volume prune</code>	Delete unused volumes (data loss!)
<code>docker network prune</code>	Delete unused networks
<code>docker builder prune</code>	Clear the build cache
<code>docker system df</code>	Show how much disk space Docker uses
<code>docker system prune</code>	Delete stopped containers, unused networks, dangling images and build cache
<code>docker system prune -a --volumes</code>	Delete everything unused, including volumes. Use with care!

10. Registry (Docker Hub)

Command	What it does
<code>docker login</code>	Log in to Docker Hub
<code>docker tag myapp username/myapp:1.0</code>	Name the image after your Docker Hub account
<code>docker push username/myapp:1.0</code>	Upload the image
<code>docker pull username/myapp:1.0</code>	Download it on another machine
<code>docker search nginx</code>	Search Docker Hub
<code>docker logout</code>	Log out

11. This project in 10 commands

```
cp .env.example .env                # 1. create your settings
docker compose up --build -d         # 2. build + start db, backend, frontend
docker compose ps                    # 3. all healthy?
docker compose logs -f backend       # 4. watch backend logs
curl http://localhost:3000/api/health # 5. test through the frontend
docker compose exec db mysql -u app -papp_password messages_db # 6. look at the DB
docker compose exec backend sh       # 7. explore inside a container
docker compose up --build -d         # 8. after changing code
docker compose down                  # 9. stop (data kept)
docker compose down -v               # 10. stop + wipe the database
```

12. Handy facts

Remember	Because
-p HOST:CONTAINER	The left side is your computer and the right side is the container. To change the URL you open, change the left side.
localhost in a container	...means the container itself. To reach another container, use its service or container name .
Containers are disposable	Anything written inside a container is lost when it's removed. Put data you want to keep in volumes .
docker compose up doesn't rebuild	After a code change, add --build , or you'll run the old image.
down vs down -v	down keeps the data and down -v deletes it.
Short IDs work	docker stop 3f2a works if the first characters of the ID are unique.